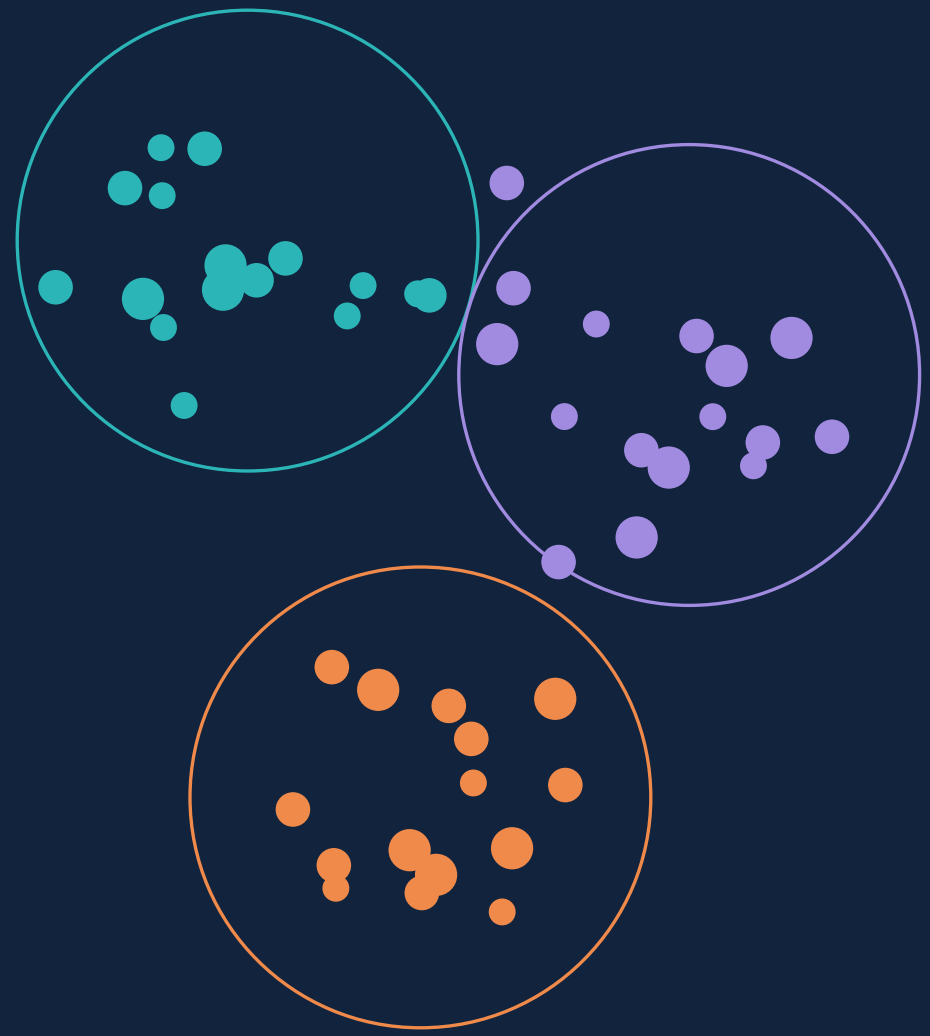
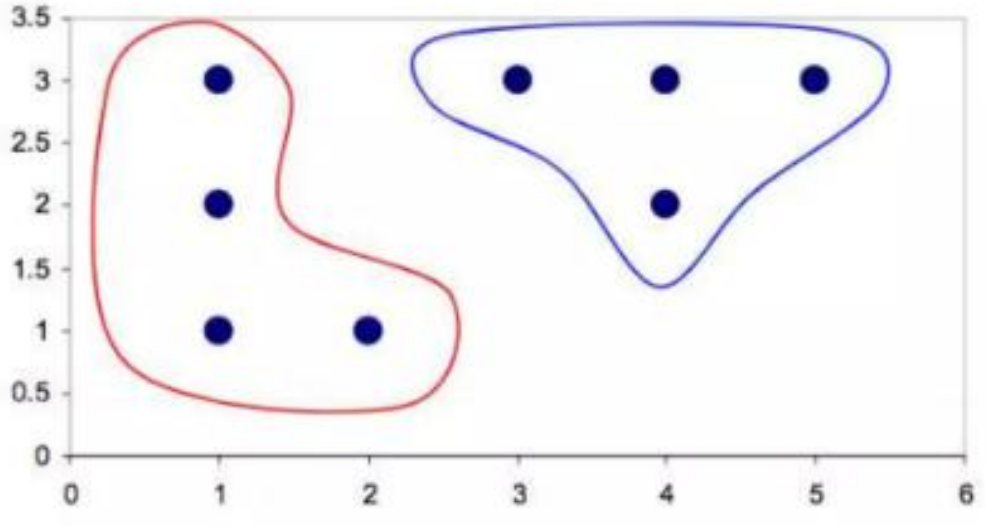
Clustering basics (Partitioned, Hierarchical and Density based)



What is Clustering?

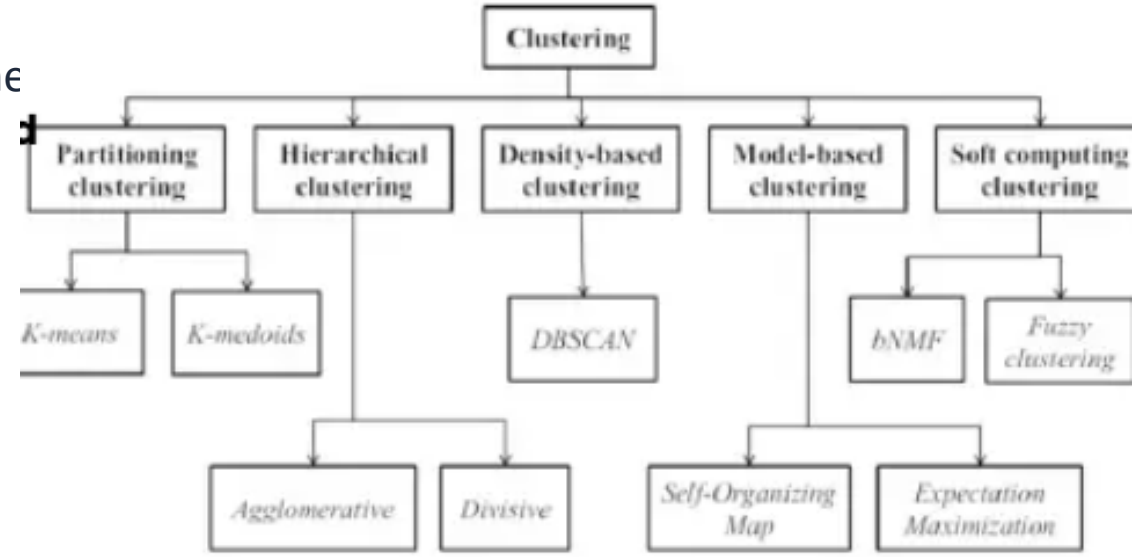
- Grouping of records ,observations or cases into classes of similar objects.
- A cluster is a collection of records,
  - Similar to one another
  - Dissimilar to records in other clusters



Introduction to Clustering

Clustering is a machine learning technique used to group similar data points together based on their characteristics, making it easier to analyze and understand large datasets. It's an unsupervised learning method where algorithms identify natural groupings without predefined labels. The main types of clustering include:-

- Partitional Clustering
- Hierarchical Clustering
- Density Based Clustering
- Model-based Clustering
- Soft computing clustering



Difference between Clustering and Classification

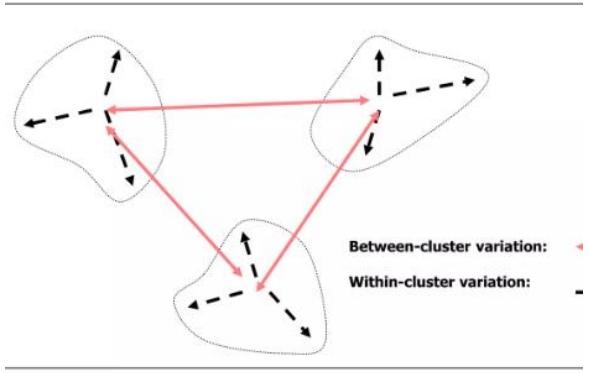
- There is **no target variable** for clustering
- Clustering does not try to classify or predict the values of a target variable.
- Instead, clustering algorithms seek to segment the entire data set into relatively homogeneous subgroups or clusters,
  - Where the similarity of the records **within the cluster is maximized**, and
  - Similarity to records **outside this cluster is minimized.**

Goal of Clustering

- Identification of groups f records such that similarity within a group is very high while the similarity to records in other groups is very low.
  - group data points that are close (or **similar**) to each other
  - identify such groupings (or clusters) in an **unsupervised** manner
    - Unsupervised: no information is provided to the algorithm on which data points belong to which clusters
- In other words,
  - Clustering algorithm seeks to construct clusters of records such that the **between-cluster variation(BCV)** is large compared to the **within-cluster variation**(WCV)

Goal of Clustering

- **(Intra-cluster distance)** the sum of distances between objects in the same cluster are **minimized**
- **(Inter-cluster distance)** while the distances between different clusters are **maximized**
- **between-cluster variation**(BCV) is large compared to the **within-cluster variation**(WCV)



Clustering

- Clustering techniques apply when there is no class to be predicted
- As we've seen clusters can be:
  - disjoint vs. overlapping
  - deterministic vs. probabilistic
  - flat vs. hierarchical
- *k-means Algorithm*
  - *k-means* clusters are disjoint, deterministic, and flat

Issues Related to Clustering

- How to measure similarity
  - Euclidian Distance
  - City-block Distance
  - Minkowski Distance
- How to recode categorical variables?
- How to standardize or normalize numerical variables?
  - Min-Max Normalization
  - Z-score standardization (m, S)
- How many clusters we expect to uncover?

Type of Clustering

- **Partitional clustering:** Partitional algorithms determine all clusters at once. They include:
  - K-Means Clustering
  - Fuzzy c-means clustering
  - QT clustering
- **Hierarchical Clustering:**
  - **Agglomerative (“bottom-up”):** Agglomerative algorithms begin with each element as a separate cluster and merge them into successively larger clusters.
  - **Divisive (“top-down”):** Divisive algorithms begin with the whole set and proceed to divide it into successively smaller clusters.

k-Means Clustering

- Input: n objects (or points) and a number k
- Algorithm

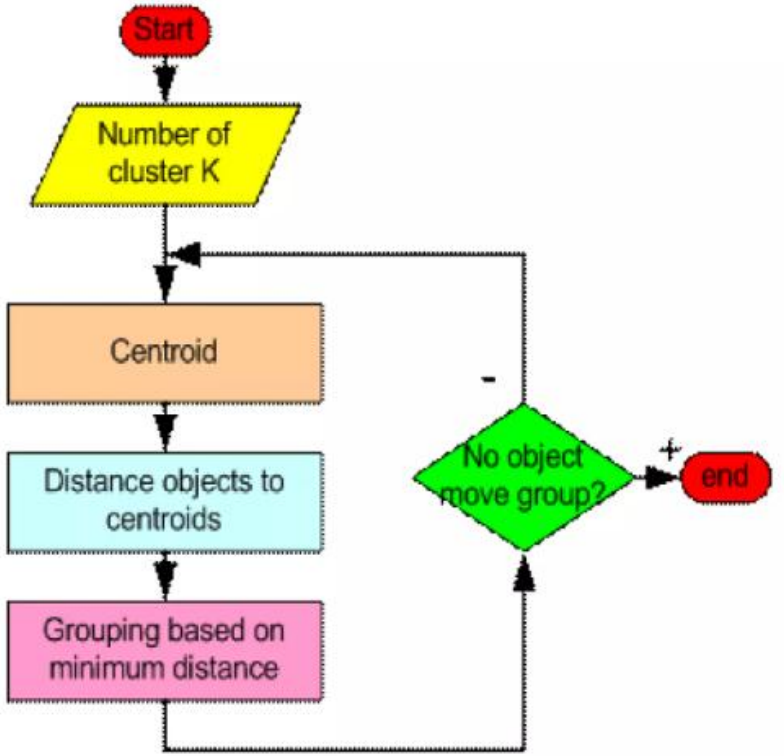
  1) Randomly assign K records to be the initial cluster center locations

  2) Assign each object to the group that has the closest centroid

  3) When all objects have been assigned, recalculate the positions of the K centroids

  4) Repeat steps 2 to 3 until convergence or termination

K-Mean Clustering



Termination Conditions

- The algorithm terminates when the centroids **no longer change.**
- The SSE(**sum of squared errors**) value is less than some small **threshold** value α

$$\mathrm{SSE} = \sum_{i=1}^{k} \sum_{p \in C_i} d(p, m_i)^2$$

- Where $p \in C_i$ represents each data point in cluster i and m_i represent the centroid of cluster i.

K-Modes Clustering Algorithm

Clustering Categorical Data

Why K-Modes?

- K-means works only on **numeric data** (uses mean and Euclidean distance)
- Categorical data such as colour, city or job has no meaningful mean
- K-modes replaces the mean with the **MODE**
- It replaces Euclidean distance with a simple matching count
- Same idea as K-means: group similar records into k clusters

Key Concepts

- **Mode**: the most frequent category of each attribute in a cluster
- **Dissimilarity**: number of attributes where two records differ
- Example: (Red, Small, Round) vs (Red, Large, Round) gives distance = 1
- **Cost**: total dissimilarity between records and their cluster modes
- **Goal:** minimise the total cost

Algorithm Steps

- 1. Choose the number of clusters k
- 2. Select k initial modes (random records)
- 3. Assign each record to the nearest mode (fewest mismatches)
- 4. Recompute the mode of every cluster
- 5. Repeat steps 3 and 4 until no record changes cluster

Problem: Grouping Students (k = 2)

Student	Sport	Music	Weekend
S1	Cricket	Rock	Outdoor
S2	Cricket	Rock	Outdoor
S3	Chess	Classical	Indoor
S4	Chess	Classical	Indoor
S5	Cricket	Classical	Outdoor
S6	Chess	Rock	Indoor

Initial modes: A = S1 (Cricket, Rock, Outdoor), B = S3 (Chess, Classical, Indoor)

Solution

- Distances: S5 to A = 1, to B = 2, so S5 joins A
- Distances: S6 to A = 2, to B = 1, so S6 joins B
- Cluster A = {S1, S2, S5}; Cluster B = {S3, S4, S6}
- New modes: A = (Cricket, Rock, Outdoor); B = (Chess, Classical, Indoor)
- Modes unchanged, so the algorithm has converged
- Total cost = 2

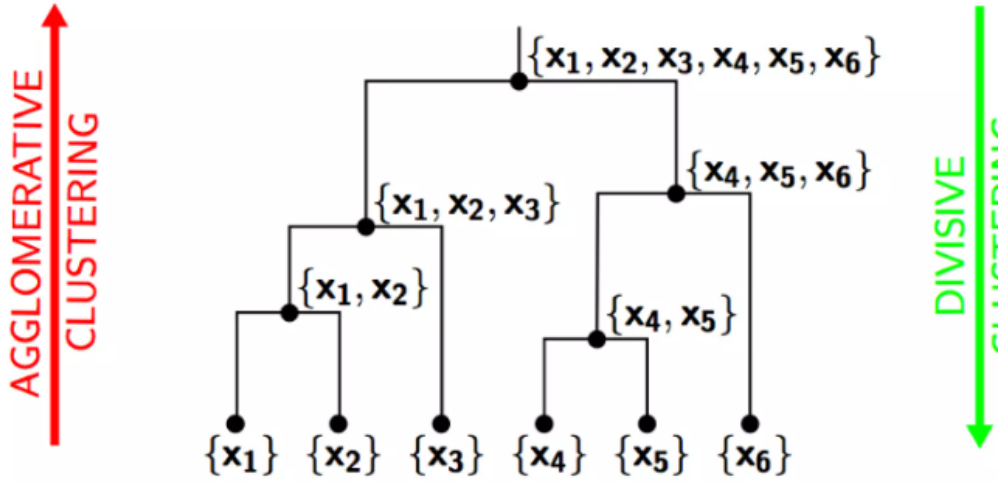
Advantages, Limitations and Uses

- Advantages: fast, simple, easy to interpret
- Limitations: sensitive to initial modes, k must be given, ties in modes
- Run several times and keep the lowest cost
- Mixed data: use K-prototypes
- Uses: customer segmentation, survey analysis, medical records

Hierarchical clustering

- There are two styles of hierarchical clustering algorithms to build a tree from the input set S:
  - **Agglomerative (bottom-up):**
    - Beginning with singletons (sets with 1 element)
    - Merging them until S is achieved as the root.
    - In each steps , the two **closest clusters** are aggregates into a new combined cluster
    - In this way, number of clusters in the data set is reduced at each step
    - Eventually, all records/elements are combined into a single huge cluster
    - It is the most common approach.
  - **Divisive (top-down):**
    - All records are combined in to a one big cluster
    - Then the most **dissimilar** records being split off recursively partitioning S until singleton sets are reached.
- Does not require the number of clusters **k** in advance

Hierarchical Clustering: Two Types



Two types of hierarchical clustering algorithms :

- Agglomerative : “bottom-up”
- Divisive : “top-down”

Classification of AHC

We can distinguish AHC algorithms according to the type of distance measures used. There are two approaches :

Graph methods :

- Single link method
- Complete link method
- Group average method (UPGMA)
- Weighted group average method (WPGMA)

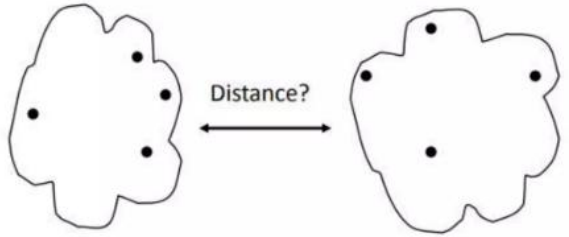
Geometric :

- Ward’s method
- Centroid method
- Median method

Agglomerative Clustering Algorithm

- More popular hierarchical clustering technique
- Basic algorithm is straightforward
  1. Compute the distance matrix
  2. Let each data point be a cluster
  3. Repeat
     4. Merge the two closest clusters
     5. Update the distance matrix
  6. Until only a single cluster remains
- Key operation is the computation of the distance between two clusters
- Different approaches to defining the distance between clusters distinguish the different algorithms

How to Define Inter-Cluster Distance



	p1	p2	p3	p4	p5	...
p1						
p2						
p3						
p4						
p5						
.						

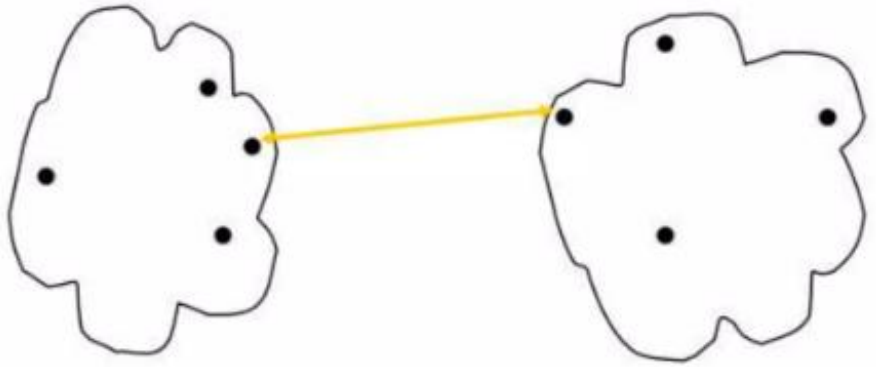
Distance Matrix

- Single link method (Min)
- Complete link method (Max)
- Average link (group Average)
- Centroid method (Distance between centriods)

Single link method (Min)

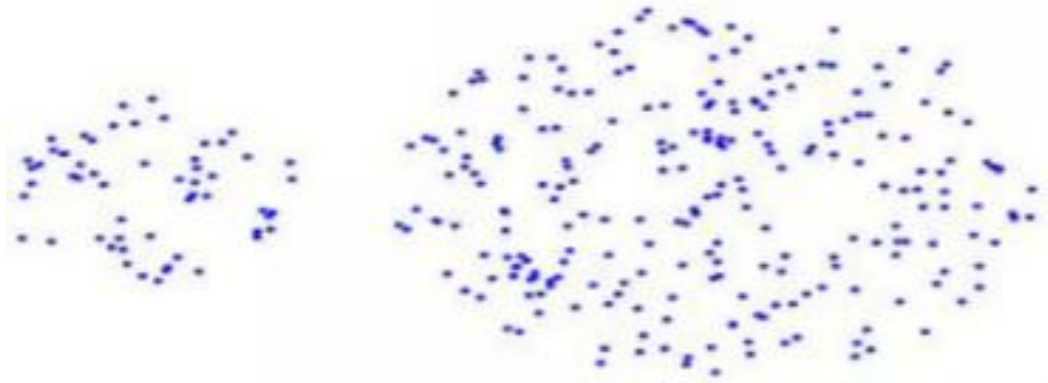
- The distance between two clusters is represented by the distance of the **closest pair of data objects** belonging to different clusters.
- Determined by one pair of points, i.e., by one link in the proximity graph

$$d_{\min}(C_i, C_j) = \min_{p \in C_\{i,\ q \in C_j\}} d(p, q)$$



Single link method (Min)

Can handle non-elliptical shapes



Original Points

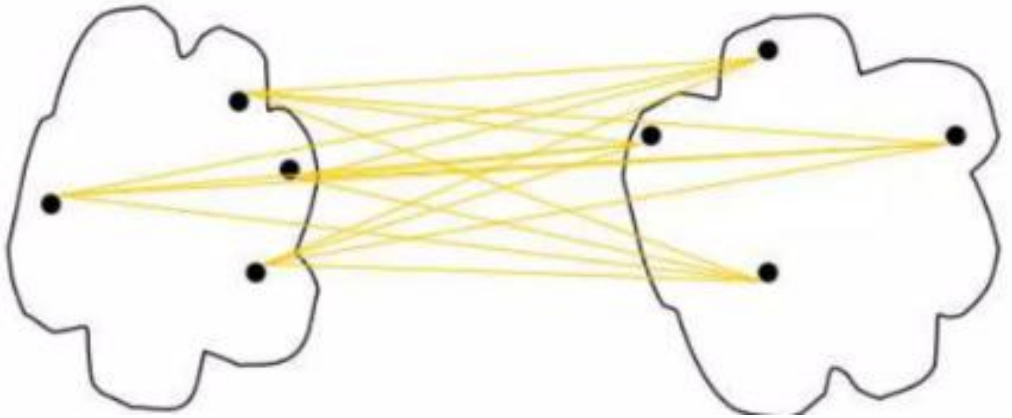


Two Clusters

Average link (Group Average)

- The distance between two clusters is represented by the average distance of all pairs of data objects belonging to different clusters
- Determined by all pairs of points in the two clusters

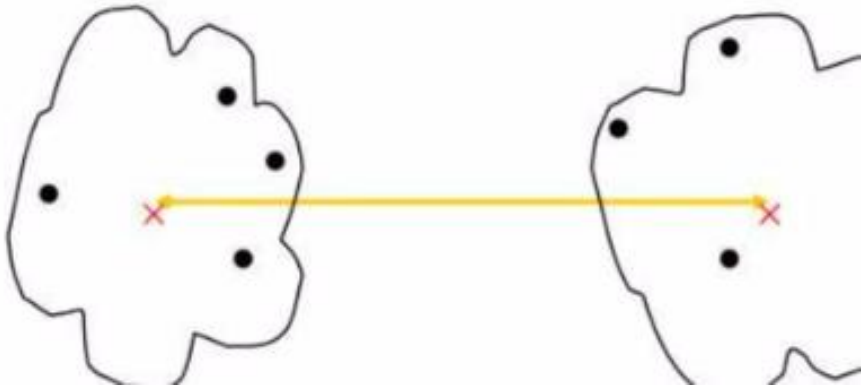
$$d_{\text{avg}}(C_i, C_j) = \text{avg}_{p \in C_\{i,\ q \in C_j\}}\ d(p, q)$$



Centroid method (Distance between centroids)

- The distance between two clusters is represented by the distance between the centers of the clusters
- Determined by cluster centroids

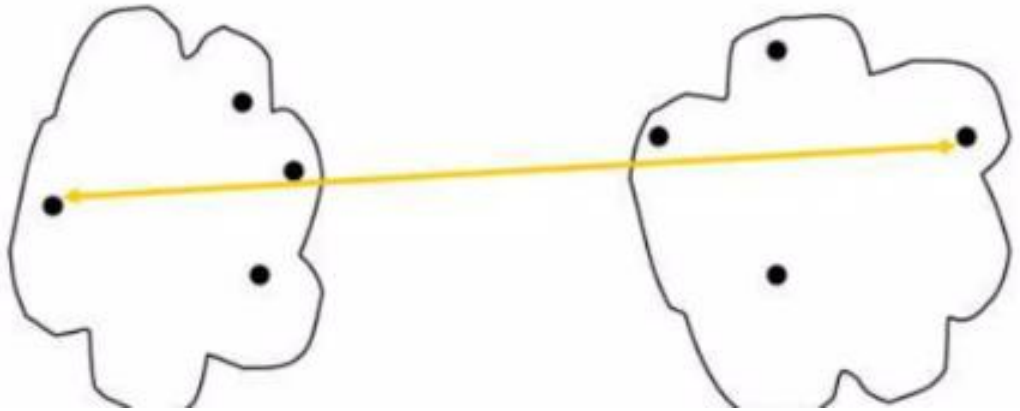
$$d_{\text{mean}}(C_i, C_j) = d(m_i, m_j)$$



Complete link method (Max)

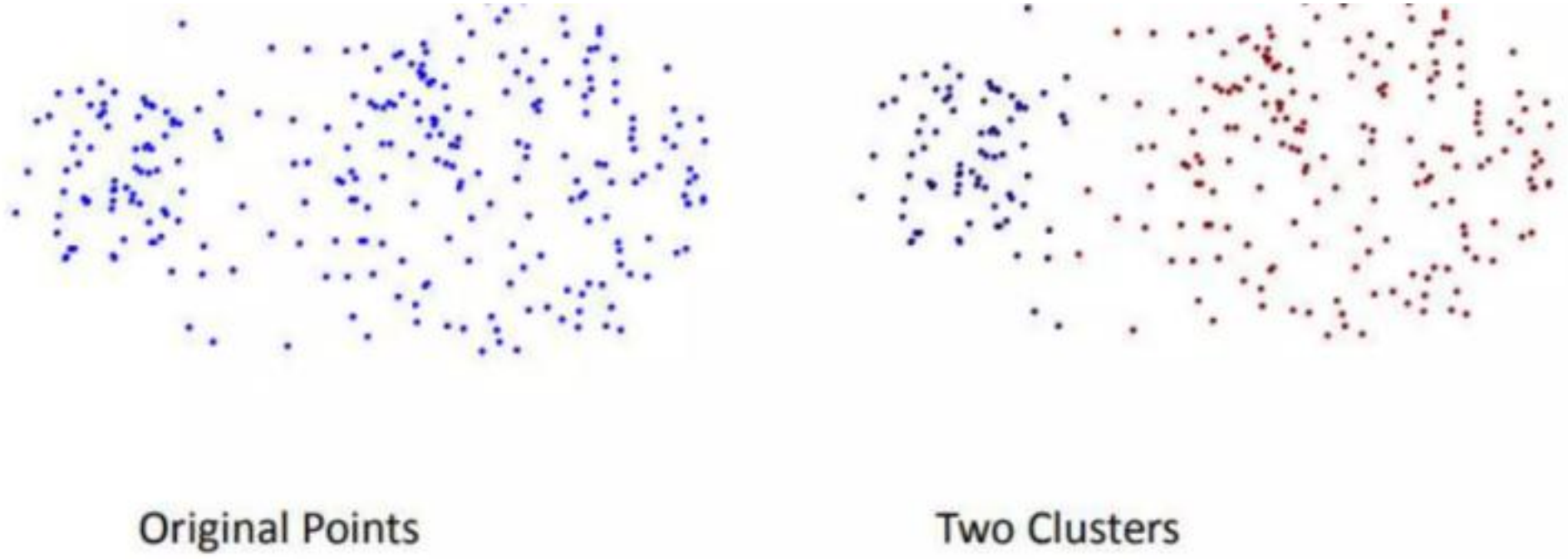
- The distance between two clusters is represented by the distance of the **farthest pair of data objects** belonging to different clusters

$$d_{\max}(C_i, C_j) = \max_{p \in C_\{i}, q \in C_j\} d(p, q)$$



Complete link method (Max)

Less susceptible to noise and outliers

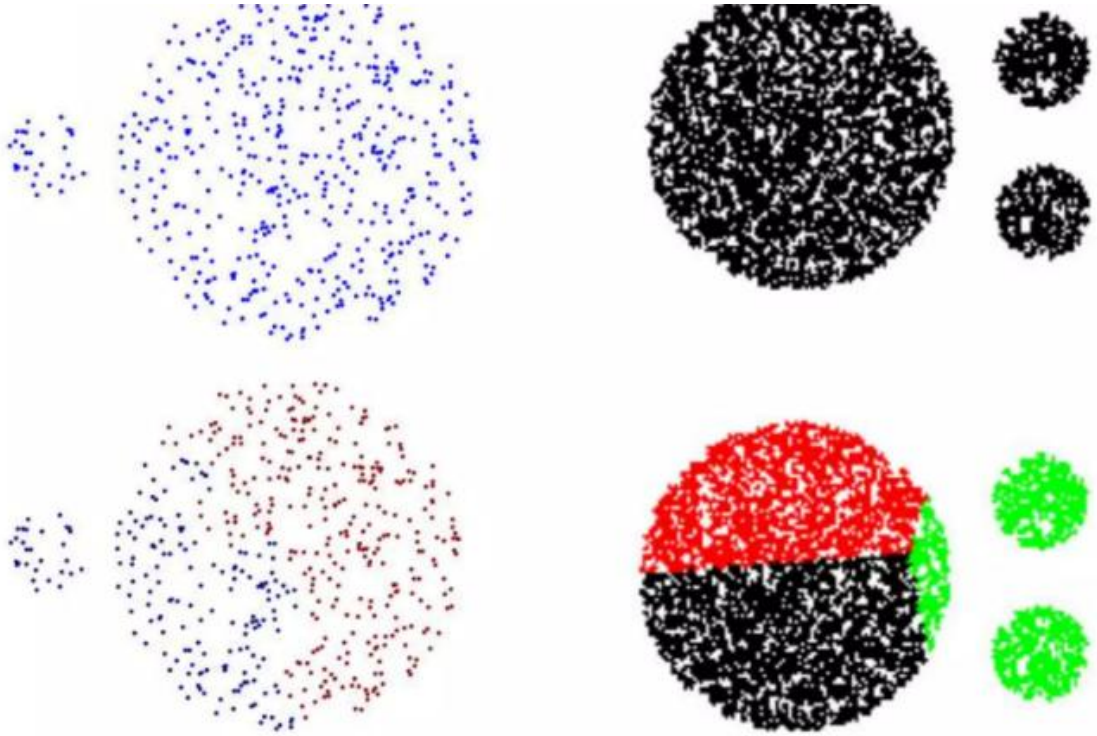


Original Points

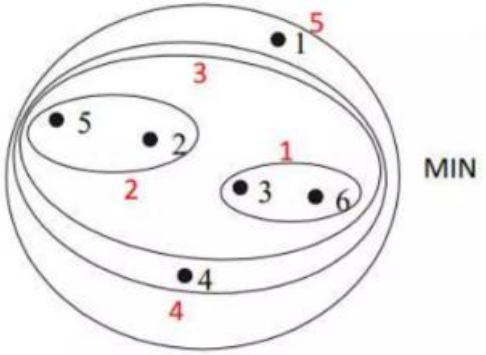
Two Clusters

Complete link method (Max)

Tends to break large clusters

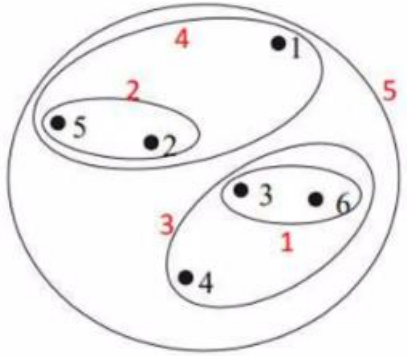


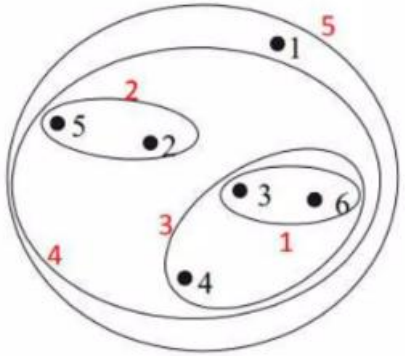
Comparison



MIN

MAX





Group Average

The Problem

Five points on a number line: A = 1, B = 2, C = 6, D = 7, E = 15



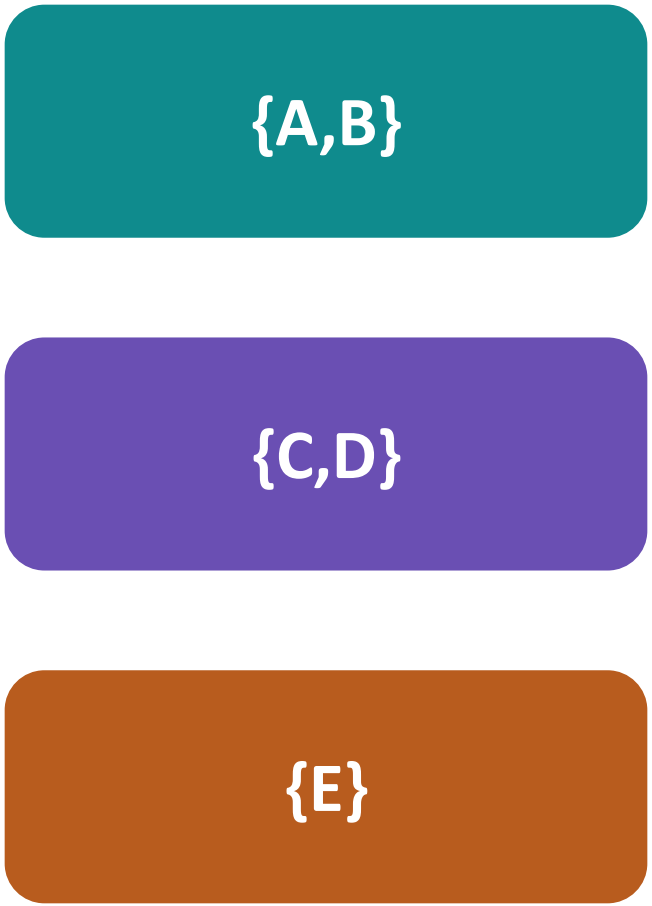
	A	B	C	D	E
A	0	1	5	6	14
B	1	0	4	5	13
C	5	4	0	1	9
D	6	5	1	0	8
E	14	13	9	8	0

Distance matrix

- Distance = difference of values
- Example: A to C = 6 - 1 = 5
- Goal: merge the closest clusters, again and again

Steps 1 and 2: Same for Every Method

- Smallest distance in the matrix is 1, and it appears twice: A-B and C-D
- Merge both, giving {A,B}, {C,D} and E
- Merge height = 1 for both merges
- Now clusters have 2 points, so we need a rule for the distance between clusters
- Each method uses a different rule - that is the only difference between them



Single Link and Complete Link

Single link (Min): closest pair

- {A,B} to {C,D}: pairs 5, 6, 4, 5 -> min = 4
- {C,D} to E: 9, 8 -> min = 8
- {A,B} to E: 14, 13 -> min = 13
- Merge {A,B} + {C,D} at height 4
- Then {A,B,C,D} to E = 8

Heights: 1, 1, 4, 8

Complete link (Max): farthest pair

- {A,B} to {C,D}: pairs 5, 6, 4, 5 -> max = 6
- {C,D} to E: 9, 8 -> max = 9
- {A,B} to E: 14, 13 -> max = 14
- Merge {A,B} + {C,D} at height 6
- Then {A,B,C,D} to E = 14

Heights: 1, 1, 6, 14

Average Link and Centroid Link

Average link: mean of all pairs

- {A,B} to {C,D}: (5+6+4+5) / 4 = 5
- {C,D} to E: (9+8) / 2 = 8.5
- {A,B} to E: (14+13) / 2 = 13.5
- Merge {A,B} + {C,D} at height 5
- Then {A,B,C,D} to E = (14+13+9+8) / 4 = 11

Heights: 1, 1, 5, 11

Centroid link: distance between means

- Centers: {A,B} = 1.5, {C,D} = 6.5, E = 15
- {A,B} to {C,D}: 6.5 - 1.5 = 5
- {C,D} to E: 15 - 6.5 = 8.5
- Merge at height 5; new center = 4
- Then {A,B,C,D} to E = 15 - 4 = 11

Heights: 1, 1, 5, 11

Comparing the Four Methods

Method	Distance rule	Merge heights	Tends to
Single	closest pair	1, 1, 4, 8	chain clusters, sensitive to noise
Complete	farthest pair	1, 1, 6, 14	compact clusters, breaks large ones
Average	mean of all pairs	1, 1, 5, 11	sit between single and complete
Centroid	distance between means	1, 1, 5, 11	be intuitive, heights can dip

Same first two merges (height 1) in every method; the methods differ from step 3 on.

Reading the Result

Dendrogram idea

- Merge heights are the heights in the dendrogram
- In all four methods E joins last, after a big jump
- That jump means E is far from everything else
- Cut just before the last merge to get 2 clusters

Final clusters

{A, B, C, D}

{E}

- E is the outlier
- Linkage changes the heights, and on messier data it can change which points group together

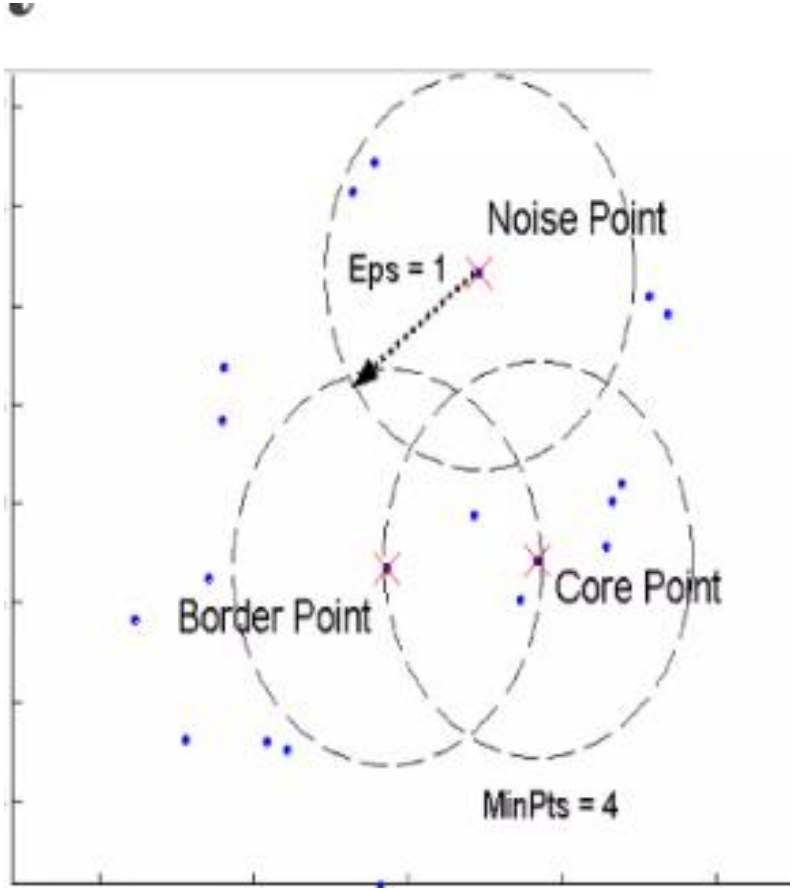
DBSACN

- *DBSCAN* is a density-based algorithm
- DBScan stands for Density-Based Spatial Clustering of Applications with Noise
- **Density-based Clustering** locates regions of high density that are separated from one another by regions of low density

Density = number of points within a specified radius (Eps)

Concepts: Preliminary

- A point is a **core point** if it has more than a specified number of points (MinPts) within Eps
  - These are points that are at the interior of a cluster
- A **border point** has fewer than MinPts within Eps, but is in the neighborhood of a core point
- A **noise point** is any point that is not a core point or a border point



Parameter Estimation

parameters must be specified by the user: ε = physical distance(radius), minPts = desired minimum cluster size

minPts

- derived from the number of dimensions D in the data set, as minPts ≥ D + 1
- minPts = 1 does not make sense, as then every point on its own will already be a cluster
- minPts must be chosen at least 3. larger is better.
- larger the data set, the larger the value of minPts should be chosen.

ε

- value can be chosen by using a k-distance graph.
- if ε is chosen much too small, a large part of the data will not be clustered.
- if too high value, majority of objects will be in the same cluster
- in general, small values of ε are preferable.

Quick Comparison of the Three Approaches

Partitional (k-means)

- Needs k in advance.
- Fast and scalable on large data.
- Best for compact, roughly round clusters; sensitive to outliers and starting centroids.

Hierarchical

- No k needed up front; cut the dendrogram later.
- Linkage choice (single, complete, average) changes the result.
- Memory grows as O(n^2), so best for small to medium data.

Density-based (DBSCAN)

- Finds arbitrary shapes and flags noise.
- Needs Eps and MinPts instead of k.
- Struggles when clusters have very different densities.